Università degli Studi di Napoli Federico II

Sistema di Gestione per un Negozio di Alimentari

Autore: Gino Pandozzi-Trani

Indice

Contents

1 Introduzione **3**

2 Analisi dei Requisiti **4**

3 Progettazione Concettuale **5**

3.1 Diagramma ER 5

3.2 Diagramma concettuale 5

3.3 Diagramma concettuale ristrutturato 7

4 Progettazione Logica **8**

4.1 Schema relazionale 8

4.2 Dizionario delle entità 8

4.3 Associazioni e relazioni 9

4.4 Vincoli di integrità 9

5 Progettazione Fisica **10**

5.1 Tabelle SQL 10

5.2 Trigger e Funzioni 14

5.3 View e Funzioni 19

6 Popolamento e Dati di Esempio **22**

1 Introduzione

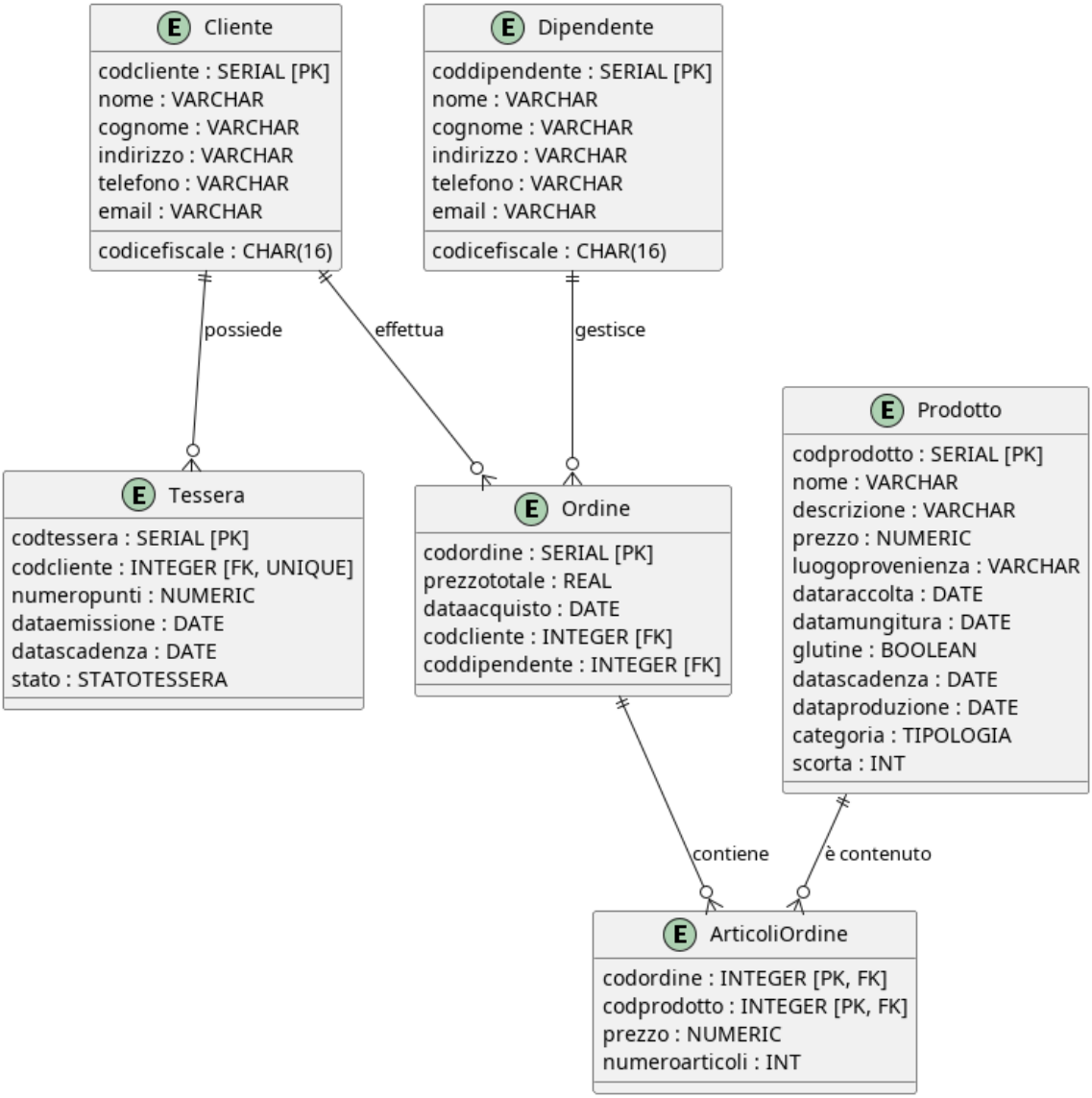
SISTEMA DI GESTIONE PER UN NEGOZIO DI ALIMENTARI Si sviluppi un sistema informativo, composto da una base di dati relazionale e un applicativo Java dotato di GUI (Swing o JavaFX), per la gestione di un sistema di un negozio di ortofrutta. Il sistema deve permettere di gestire le vendite e la disponibilità dei prodotti disponibili, identificando le tipologie (frutta, verdura, farinacei, latticini, uova e confezionati). Ogni tipologia di prodotto ha delle caratteristiche diverse (ad esempio, la frutta e la verdura hanno una data di raccolta, i latticini hanno sia una data di mungitura del latte che una data di produzione, ecc..). Ogni cliente è registrato e ha una tessera punti associata che lo identifica al fruttivendolo. Per ogni acquisto il cliente riceve il 10% del valore della spesa in punti fedeltà. Il sistema deve premettere la ricerca dei clienti, differenziandoli sulla base delle categorie di prodotti acquistati e sulla quantità di punti che hanno ottenuto per ogni categoria. Per il gruppo da 3: il fruttivendolo ha un certo numero di dipendenti, e ad ogni vendita è associato il dipendente che l'ha portata a termine. Deve pertanto essere possibile ricercare quale dipendente ha effettuato il maggior numero di vendite in un determinato periodo differenziandolo dal dipendente che ha portato al fruttivendolo il maggior introito. Breve presentazione del progetto, obiettivi e contesto della traccia accademica.

2 Analisi dei Requisiti

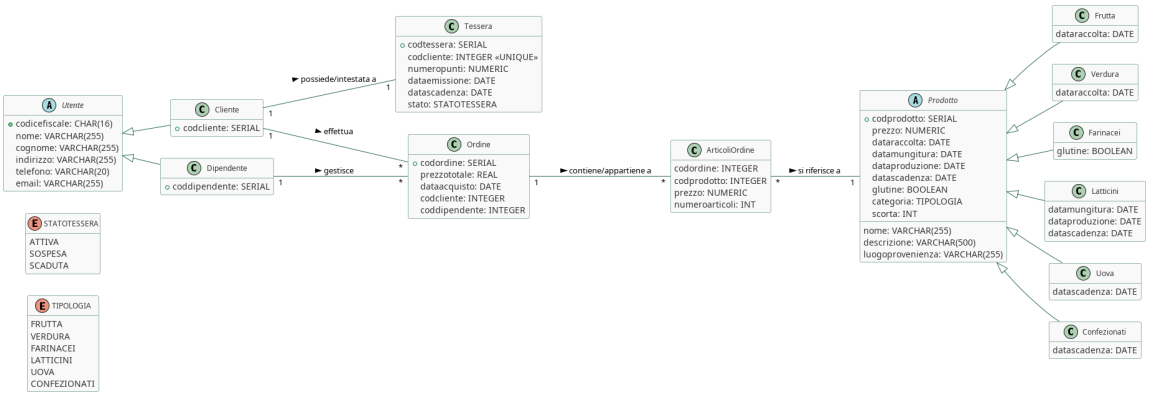
- Gestione prodotti per tipologia (frutta, verdura, farinacei, latticini, uova, confezionati)
- Gestione clienti e tessere punti
- Gestione dipendenti e vendite
- Statistiche e ricerche

3 Progettazione Concettuale

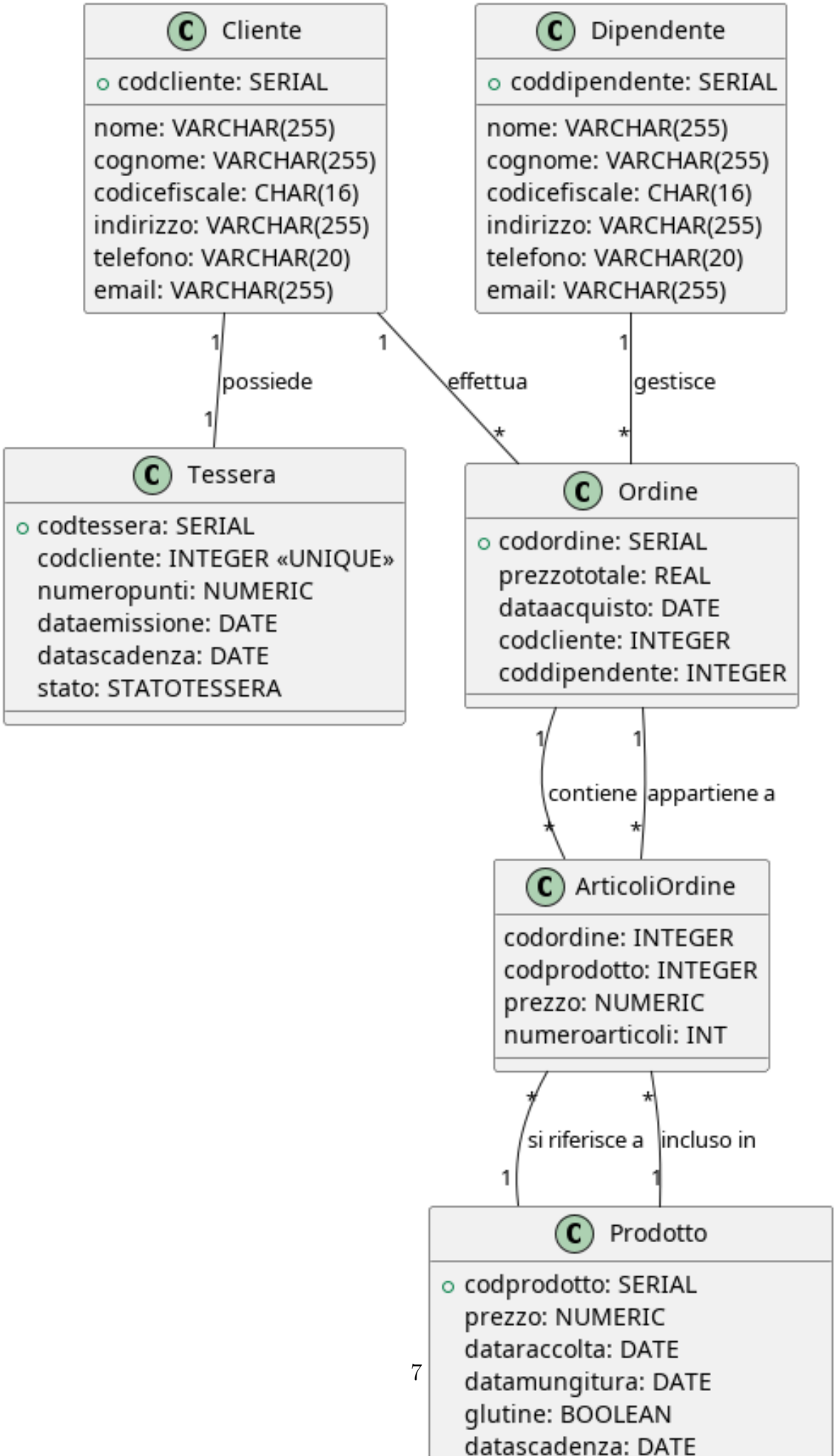
3.1 Diagramma ER



3.2 Diagramma concettuale



3.3 Diagramma concettuale ristrutturato



4 Progettazione Logica

4.1 Schema relazionale

Lo schema relazionale è stato progettato per riflettere fedelmente le esigenze di gestione di un negozio di alimentari. Sono stati definiti tipi enumerativi per le categorie di prodotto (`TIPOLOGIA`) e per lo stato della tessera fedeltà (`STATOTESSERA`), assicurando la validazione dei dati e la chiarezza semantica.

Le principali entità (cliente, tessera, dipendente, prodotto, ordine, articoliordine) sono state implementate con vincoli di unicità, referenzialità e regole di business. In particolare, la tabella `prodotto` include un vincolo `CHECK` che garantisce la coerenza degli attributi in base alla categoria, evitando errori di inserimento e mantenendo la qualità dei dati.

Tabella	Attributi
cliente	codcliente (PK), nome, cognome, codicefiscale, indirizzo, telefono, email
tessera	codtessera (PK), codcliente (FK, UNIQUE), numeropunti, dataemissione, datascadenza, stato
dipendente	coddipendente (PK), nome, cognome, codicefiscale, indirizzo, telefono, email
prodotto	codprodotto (PK), nome, descrizione, prezzo, luogoprovenienza, dataraccolta, datamungitura, glutine, datascadenza, dataproduzione, categoria, scorta
ordine	codordine (PK), prezzototale, dataacquisto, codcliente (FK), coddipendente (FK)
articoliordine	codordine (PK, FK), codprodotto (PK, FK), prezzo, numeroarticoli

Table 1: Schema relazionale: chiavi primarie sottolineate, chiavi esterne doppiamente sottolineate.

4.2 Dizionario delle entità

Ogni tabella è stata pensata per rappresentare un aspetto fondamentale del sistema: i clienti e i dipendenti sono identificati in modo univoco e validati tramite espressioni regolari per email e codice fiscale; la tessera fedeltà è associata a ciascun cliente e gestisce il saldo punti e lo stato; i prodotti sono caratterizzati da attributi specifici per tipologia; gli ordini e gli articoli d'ordine permettono di tracciare le transazioni e i movimenti di magazzino.

Entità (Tabella)	Descrizione
cliente	Dati anagrafici dei clienti registrati nel sistema.
tessera	Tessera punti associata a ciascun cliente, saldo punti e stato.
dipendente	Dati anagrafici dei dipendenti del negozio.
prodotto	Prodotti in vendita, con tipologia, caratteristiche e scorta.
ordine	Vendita, associata a cliente e dipendente.
articoliordine	Dettaglio dei prodotti acquistati in ogni ordine.

Table 2: Entità principali del database e loro funzione.

4.3 Associazioni e relazioni

Le relazioni tra le entità sono state implementate tramite chiavi esterne e vincoli di integrità referenziale. Il sistema supporta associazioni uno-a-uno (cliente-tessera), uno-a-molti (cliente-ordine, dipendente-ordine) e molti-a-molti (ordine-prodotto tramite articoliordine), garantendo la tracciabilità e la consistenza delle operazioni.

Associazione	Descrizione
Cliente-Tessera	Ogni cliente ha una tessera (1:1).
Ordine-Cliente	Ogni ordine è associato a un cliente (N:1).
Ordine-Dipendente	Ogni ordine è associato a un dipendente (N:1).
Ordine-Prodotto	Ogni ordine contiene più prodotti tramite articoliordine (N:M).

Table 3: Principali associazioni tra le entità del database.

4.4 Vincoli di integrità

Oltre ai vincoli di chiave primaria ed esterna, sono stati introdotti vincoli `CHECK` su valori numerici (ad esempio, scorta e prezzo non negativi) e regole di validazione per i dati obbligatori. Il vincolo sulla tabella `prodotto` assicura che ogni categoria abbia solo gli attributi pertinenti valorizzati, riducendo il rischio di errori e facilitando la gestione.

Vincolo	Descrizione
Email Regex	Gli indirizzi email di clienti e dipendenti devono essere di forma legittima.
Codice Fiscale Regex	Il codice fiscale deve rispettare il formato italiano standard.
Chiavi Primarie	Ogni tabella ha una chiave primaria che garantisce l'unicità dei record.
Chiavi Esterne	Le chiavi esterne garantiscono l'integrità referenziale tra le tabelle.
Vincoli CHECK	Su valori numerici (es. scorta ≥ 0, prezzo ≥ 0), presenza di dati obbligatori, coerenza attributi per categoria prodotto.

Table 4: Vincoli e regole di integrità implementati nel database.

5 Progettazione Fisica

La progettazione fisica rappresenta la fase in cui il modello logico viene tradotto in strutture concrete all'interno del DBMS scelto. In questa sezione vengono presentati gli script SQL che permettono la creazione delle tabelle, dei vincoli di integrità, delle funzioni, dei trigger e delle viste, garantendo la robustezza, la sicurezza e l'efficienza del sistema informativo. L'obiettivo è fornire una base dati pronta all'uso, ottimizzata per le operazioni di gestione e analisi tipiche di un negozio di alimentari.

5.1 Tabelle SQL

Di seguito sono riportati le tabelle SQL necessarie per la creazione delle principali strutture del database: tipi enumerativi, tabelle, vincoli e relazioni. Ogni script è stato progettato per assicurare la coerenza dei dati e la corretta rappresentazione delle regole di business individuate in fase di analisi.

Enumerazioni

```sql
-- Creare il nuovo tipo ENUM con le categorie corrette e
   migliorato
CREATE TYPE TIPOLOGIA AS ENUM (
    'FRUTTA',
    'VERDURA',
    'FARINACEI',
    'LATTICINI',
    'UOVA',
    'CONFEZIONATI'
);

-- Creare tipo ENUM per lo stato della tessera
CREATE TYPE STATOTESSERA AS ENUM (
    'ATTIVA',
```

```
    'SOSPESA',
    'SCADUTA'
);
```

Tabella cliente

```
CREATE TABLE IF NOT EXISTS cliente (
    codcliente SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL CHECK (nome ~* '^[A-Za-z ]+$'),
    cognome VARCHAR(255) NOT NULL CHECK (cognome ~* '^[A-Za-z ]+$
       '),
    codicefiscale CHAR(16) NOT NULL UNIQUE CHECK (codicefiscale
       ~* '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$'),
    indirizzo VARCHAR(255) NOT NULL,
    telefono VARCHAR(20) CHECK (telefono ~* '^[0-9+ ]+$'),
    email VARCHAR(255) NOT NULL UNIQUE CHECK (email ~* '^[A-Za-z0
       -9._%-]+@[A-Za-z0-9.-]+\.[A-Za-z]+$')
);
```

Tabella tessera

```
CREATE TABLE IF NOT EXISTS tessera (
    codtessera SERIAL PRIMARY KEY,
    codcliente INTEGER NOT NULL UNIQUE,
    numeropunti NUMERIC NOT NULL DEFAULT 0.00 CHECK (numeropunti
       >= 0.00),
    dataemissione DATE NOT NULL DEFAULT CURRENT_DATE,
    datascadenza DATE NOT NULL DEFAULT (CURRENT_DATE + INTERVAL '
       2 years'),
    stato STATOTESSERA NOT NULL DEFAULT 'ATTIVA',
    CONSTRAINT tessera_cliente_fk FOREIGN KEY (codcliente)
       REFERENCES cliente (codcliente)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

Tabella dipendente

```
CREATE TABLE IF NOT EXISTS dipendente (
    coddipendente SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL CHECK (nome ~* '^[A-Za-z ]+$'),
```

```
    cognome VARCHAR(255) NOT NULL CHECK (cognome ~* '^[A-Za-z ]+$
        '),
    codicefiscale CHAR(16) NOT NULL UNIQUE CHECK (codicefiscale
        ~* '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$'),
    indirizzo VARCHAR(255) NOT NULL,
    telefono VARCHAR(20) CHECK (telefono ~* '^[0-9+ ]+$'),
    email VARCHAR(255) NOT NULL UNIQUE CHECK (email ~* '^[A-Za-z0
        -9._%-]+@[A-Za-z0-9.-]+\.[A-Za-z]+$')
);
```

Tabella prodotto

```
CREATE TABLE IF NOT EXISTS prodotto (
    codprodotto SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL CHECK (nome ~* '^[A-Za-z ]+$'),
    descrizione VARCHAR(500),
    prezzo NUMERIC DEFAULT 0.00 CHECK (prezzo >= 0.00),
    luogoprovenienza VARCHAR(255),
    dataraccolta DATE,
    datamungitura DATE,
    glutine BOOLEAN,
    datascadenza DATE,
    dataproduzione DATE,
    categoria TIPOLOGIA,
    scorta INT CHECK (scorta >= 0)
);
```

Vincolo CHECK su prodotto

```
ALTER TABLE prodotto ADD CONSTRAINT checkCategoria CHECK (
  -- FRUTTA: deve avere data di raccolta
  (categoria = 'FRUTTA' AND dataraccolta IS NOT NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NULL AND glutine IS NULL) OR

  -- VERDURA: deve avere data di raccolta
  (categoria = 'VERDURA' AND dataraccolta IS NOT NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NULL AND glutine IS NULL) OR

  -- LATTICINI: devono avere data di mungitura del latte e data
     di produzione
  (categoria = 'LATTICINI' AND dataraccolta IS NULL AND
     datamungitura IS NOT NULL AND dataproduzione IS NOT NULL AND
     datascadenza IS NOT NULL AND glutine IS NULL) OR
```

```
  -- FARINACEI: informazioni sul glutine
  (categoria = 'FARINACEI' AND dataraccolta IS NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NULL AND glutine IS NOT NULL) OR

  -- UOVA: solo data di scadenza
  (categoria = 'UOVA' AND dataraccolta IS NULL AND datamungitura
     IS NULL AND dataproduzione IS NULL AND datascadenza IS NOT
     NULL AND glutine IS NULL) OR

  -- CONFEZIONATI: solo data di scadenza
  (categoria = 'CONFEZIONATI' AND dataraccolta IS NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NOT NULL AND glutine IS NULL)
);
```

Tabella ordine

```
CREATE TABLE IF NOT EXISTS ordine (
    codordine SERIAL PRIMARY KEY,
    prezzototale REAL NOT NULL DEFAULT 0.00 CHECK (prezzototale
       >= 0),
    dataacquisto DATE NOT NULL,
    codcliente INTEGER NOT NULL,
    coddipendente INTEGER NOT NULL,
    CONSTRAINT ordineclientefk FOREIGN KEY (codcliente)
       REFERENCES cliente (codcliente)
        ON UPDATE CASCADE
        ON DELETE NO ACTION,
    CONSTRAINT ordinedipendentefk FOREIGN KEY (coddipendente)
       REFERENCES dipendente (coddipendente)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);
```

Tabella articoliordine

```
CREATE TABLE IF NOT EXISTS articoliordine (
    codordine INTEGER NOT NULL,
    codprodotto INTEGER NOT NULL,
    prezzo NUMERIC NOT NULL DEFAULT 0.00 CHECK (prezzo >= 0.00),
    numeroarticoli INT NOT NULL CHECK (numeroarticoli > 0),
    CONSTRAINT PK_ArticoliOrdine PRIMARY KEY (codordine,
       codprodotto),
```

```
    CONSTRAINT articoliordineprodottofk FOREIGN KEY (codprodotto)
        REFERENCES prodotto (codprodotto)
        ON UPDATE CASCADE
        ON DELETE NO ACTION,
    CONSTRAINT articoliordineordinek FOREIGN KEY (codordine)
       REFERENCES ordine (codordine)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

5.2 Trigger e Funzioni

In questa sezione vengono presentati i principali trigger e funzioni che automatizzano la gestione dei dati e garantiscono il rispetto delle regole di integrità. I trigger agiscono automaticamente su eventi specifici (inserimenti, aggiornamenti, cancellazioni), mentre le funzioni eseguono calcoli e controlli complessi, migliorando la qualità e la sicurezza delle operazioni.

Nota Importante

L'utilizzo di trigger e funzioni consente di:

- Aggiornare automaticamente la scorta dei prodotti e i punti fedeltà dei clienti.
- Verificare la disponibilità dei prodotti prima di ogni vendita.
- Mantenere la coerenza tra ordini, articoli e tessere punti.
- Prevenire errori e garantire la robustezza del sistema.

Panoramica dei Trigger

Di seguito una panoramica dei principali trigger implementati, con una breve descrizione del loro scopo:

- **TrgAggiornaScortaPunti**: aggiorna la scorta del prodotto e i punti della tessera cliente dopo l'inserimento di un articolo in un ordine.
- **TrgAggiornaPrezzoOrdine**: aggiorna il prezzo totale dell'ordine dopo l'inserimento di un articolo.
- **TrgVerificaScortaProdotto**: verifica che la scorta sia sufficiente prima di inserire un articolo nell'ordine.
- **TrgAggiornaStatoTessera**: aggiorna lo stato della tessera (es. scaduta) in base alla data di scadenza.
- **TrgRipristinaScortaPunti**: ripristina la scorta e aggiorna i punti tessera dopo la cancellazione di un articolo da un ordine.

- **TrgAggiornaPrezzoOrdineDelete**: aggiorna il prezzo totale dell'ordine dopo la cancellazione di un articolo.

Esempio pratico:

Quando un cliente acquista 3 mele (prodotto con scorta 10), il trigger `TrgAggiornaScortaPunti` riduce la scorta a 7 e aggiunge il 10% del valore speso ai punti della tessera associata. Se la scorta fosse insufficiente, il trigger `TrgVerificaScortaProdotto` bloccherebbe l'operazione.

Trigger principali

```
-- Trigger per aggiornare scorta e punti tessera dopo inserimento
    articolo
CREATE TRIGGER TrgAggiornaScortaPunti
AFTER INSERT ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION AggiornaScortaEPunti();

-- Trigger per aggiornare il prezzo totale dell'ordine dopo
   inserimento articolo
CREATE TRIGGER TrgAggiornaPrezzoOrdine
AFTER INSERT ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION AggiornaPrezzoOrdine();

-- Trigger per verificare la scorta prima dell'inserimento
CREATE TRIGGER TrgVerificaScortaProdotto
BEFORE INSERT ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION VerificaScortaProdotto();

-- Trigger per aggiornare lo stato della tessera
CREATE TRIGGER TrgAggiornaStatoTessera
BEFORE INSERT OR UPDATE ON tessera
FOR EACH ROW
EXECUTE FUNCTION AggiornaStatoTessera();

-- Trigger per ripristinare scorta e punti tessera dopo DELETE da
    articoliordine
CREATE TRIGGER TrgRipristinaScortaPunti
AFTER DELETE ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION RipristinaScortaProdotto();

-- Trigger per aggiornare il prezzo totale dell'ordine dopo
   DELETE da articoliordine
CREATE TRIGGER TrgAggiornaPrezzoOrdineDelete
```

```
AFTER DELETE ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION AggiornaPrezzoOrdineDelete();
```

Descrizione delle Funzioni

Ogni funzione associata ai trigger svolge un compito specifico. Di seguito una breve spiegazione prima del codice:

- **AggiornaScortaEPunti**: aggiorna la scorta del prodotto e incrementa i punti della tessera cliente in base al valore speso.
- **AggiornaPrezzoOrdine**: aggiorna il totale dell'ordine dopo l'inserimento di un articolo.
- **VerificaScortaProdotto**: controlla che la scorta sia sufficiente prima di confermare la vendita.
- **AggiornaStatoTessera**: imposta lo stato della tessera a "SCADUTA" se la data di scadenza è superata.
- **RipristinaScortaProdotto**: ripristina la scorta e aggiorna i punti tessera dopo la cancellazione di un articolo.
- **AggiornaPrezzoOrdineDelete**: aggiorna il totale dell'ordine dopo la cancellazione di un articolo.

Per ogni funzione, il codice è preceduto da una breve descrizione e, dove utile, da un esempio pratico di utilizzo.

Funzione AggiornaScortaEPunti

```
CREATE OR REPLACE FUNCTION AggiornaScortaEPunti()
RETURNS TRIGGER AS $$
DECLARE
    punti_da_aggiungere NUMERIC;
    cod_cliente INTEGER;
BEGIN
    -- Aggiorna la scorta del prodotto
    UPDATE prodotto
    SET scorta = scorta - NEW.numeroarticoli
    WHERE codprodotto = NEW.codprodotto;

    -- Calcola i punti da aggiungere (10% del valore speso)
    punti_da_aggiungere := NEW.prezzo * NEW.numeroarticoli *
       0.10;

    -- Recupera il cliente associato all'ordine
```

```
    SELECT codcliente INTO cod_cliente
    FROM ordine
    WHERE codordine = NEW.codordine;

    -- Aggiorna i punti della tessera del cliente
    UPDATE tessera
    SET numeropunti = numeropunti + punti_da_aggiungere
    WHERE codcliente = cod_cliente;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione AggiornaPrezzoOrdine

```
CREATE OR REPLACE FUNCTION AggiornaPrezzoOrdine()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE ordine
    SET prezzototale = prezzototale + (NEW.prezzo * NEW.
       numeroarticoli)
    WHERE codordine = NEW.codordine;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione VerificaScortaProdotto

```
CREATE OR REPLACE FUNCTION VerificaScortaProdotto()
RETURNS TRIGGER AS $$
DECLARE
    scorta_corrente INT;
BEGIN
    SELECT scorta INTO scorta_corrente FROM prodotto WHERE
       codprodotto = NEW.codprodotto;
    IF scorta_corrente < NEW.numeroarticoli THEN
        RAISE EXCEPTION 'Scorta insufficiente per il prodotto %:
           richiesta % - disponibile %', NEW.codprodotto, NEW.
           numeroarticoli, scorta_corrente;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione AggiornaStatoTessera

```
CREATE OR REPLACE FUNCTION AggiornaStatoTessera()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.datascadenza < CURRENT_DATE THEN
        NEW.stato := 'SCADUTA';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione RipristinaScortaProdotto

```
CREATE OR REPLACE FUNCTION RipristinaScortaProdotto()
RETURNS TRIGGER AS $$
DECLARE
    cod_cliente INTEGER;
    punti_da_togliere NUMERIC;
BEGIN
    -- Ripristina la scorta del prodotto
    UPDATE prodotto
    SET scorta = scorta + OLD.numeroarticoli
    WHERE codprodotto = OLD.codprodotto;

    -- Recupera il cliente associato all'ordine
    SELECT codcliente INTO cod_cliente
    FROM ordine
    WHERE codordine = OLD.codordine;

    -- Calcola i punti da togliere (10% del valore speso)
    punti_da_togliere := OLD.prezzo * OLD.numeroarticoli * 0.10;

    -- Aggiorna i punti della tessera del cliente
    UPDATE tessera
    SET numeropunti = GREATEST(numeropunti - punti_da_togliere,
       0)
    WHERE codcliente = cod_cliente;

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

Funzione AggiornaPrezzoOrdineDelete

```
CREATE OR REPLACE FUNCTION AggiornaPrezzoOrdineDelete()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE ordine
    SET prezzototale = GREATEST(prezzototale - (OLD.prezzo * OLD.
       numeroarticoli), 0)
    WHERE codordine = OLD.codordine;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

5.3 View e Funzioni

Le viste e le funzioni presentate in questa sezione consentono di semplificare le interrogazioni e di ottenere rapidamente informazioni aggregate e dettagliate. Le view facilitano la consultazione dei dati più rilevanti per la gestione del negozio, mentre le funzioni parametrizzate permettono di eseguire ricerche e statistiche avanzate, rispondendo alle esigenze di analisi e reportistica.

```
CREATE OR REPLACE FUNCTION RicercaClientiPerCategoria(
   categoriaRicerca TIPOLOGIA DEFAULT NULL)
RETURNS TABLE(
    codcliente INTEGER,
    nome VARCHAR(255),
    cognome VARCHAR(255),
    categoria TIPOLOGIA,
    punti_categoria NUMERIC,
    spesa_categoria NUMERIC,
    ordini_categoria BIGINT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.codcliente,
        c.nome,
        c.cognome,
        p.categoria,
        ROUND(COALESCE(SUM(ao.prezzo * ao.numeroarticoli * 0.10),
            0), 2) as punti_categoria,
        ROUND(COALESCE(SUM(ao.prezzo * ao.numeroarticoli), 0), 2)
            as spesa_categoria,
        COUNT(DISTINCT o.codordine)::BIGINT as ordini_categoria
    FROM cliente c
    JOIN ordine o ON c.codcliente = o.codcliente
    JOIN articoliordine ao ON o.codordine = ao.codordine
    JOIN prodotto p ON ao.codprodotto = p.codprodotto
```

```
    WHERE (categoriaRicerca IS NULL OR p.categoria =
       categoriaRicerca)
    GROUP BY c.codcliente, c.nome, c.cognome, p.categoria
    HAVING COALESCE(SUM(ao.prezzo * ao.numeroarticoli * 0.10), 0)
        > 0
    ORDER BY c.codcliente, punti_categoria DESC;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION DipendenteStatistiche(data_inizio DATE
   , data_fine DATE)
RETURNS TABLE(nome TEXT, cognome TEXT, NumeroOrdini INTEGER,
   Introiti NUMERIC)
AS $$
    SELECT nome, cognome, NumeroOrdini, NULL AS Introiti
    FROM (
        SELECT D.nome, D.cognome, COUNT(O.*) AS NumeroOrdini
        FROM dipendente AS D
        JOIN ordine AS O ON O.coddipendente = D.coddipendente
        WHERE O.dataacquisto BETWEEN data_inizio AND data_fine
        GROUP BY D.nome, D.cognome
        ORDER BY NumeroOrdini DESC
        LIMIT 1
    ) AS TopVendite
    UNION ALL
    SELECT nome, cognome, NULL AS NumeroOrdini, Introiti
    FROM (
        SELECT D.nome, D.cognome, SUM(O.prezzototale) AS Introiti
        FROM dipendente AS D
        JOIN ordine AS O ON O.coddipendente = D.coddipendente
        WHERE O.dataacquisto BETWEEN data_inizio AND data_fine
        GROUP BY D.nome, D.cognome
        ORDER BY Introiti DESC
        LIMIT 1
    ) AS TopIntroito;
$$ LANGUAGE sql;
```

```
CREATE OR REPLACE VIEW VwClientiTessere AS
SELECT c.codcliente, c.nome, c.cognome, t.numeropunti, t.stato, t
   .datascadenza
FROM cliente c
JOIN tessera t ON c.codcliente = t.codcliente;
```

```
CREATE OR REPLACE VIEW VwProdottiScortaBassa AS
SELECT codprodotto, nome, categoria, scorta
FROM prodotto
WHERE scorta < 20;
```

```
CREATE OR REPLACE VIEW VwOrdiniDettagliati AS
SELECT o.codordine, o.dataacquisto, o.prezzototale, c.nome AS
   nome_cliente, c.cognome AS cognome_cliente,
```

```sql
        d.nome AS nome_dipendente, d.cognome AS cognome_dipendente
            ,
        p.nome AS nome_prodotto, p.categoria, ao.numeroarticoli,
            ao.prezzo
FROM ordine o
JOIN cliente c ON o.codcliente = c.codcliente
JOIN dipendente d ON o.coddipendente = d.coddipendente
JOIN articoliordine ao ON o.codordine = ao.codordine
JOIN prodotto p ON ao.codprodotto = p.codprodotto;
```

6 Popolamento e Dati di Esempio

Il popolamento del database è stato effettuato tramite lo script `Popolazione.sql`, che contiene tutti gli **INSERT** necessari per popolare le tabelle con dati di esempio realistici e coerenti con la struttura progettata. Questo file è fondamentale per testare il funzionamento delle query, dei trigger e delle funzioni implementate, simulando un ambiente operativo completo e credibile. **Cosa contiene Popolazione.sql?**

Lo script inserisce prodotti di tutte le tipologie (frutta, verdura, farinacei, latticini, uova, confezionati), ciascuno con attributi specifici per categoria. Vengono aggiunti clienti e relative tessere punti con saldo iniziale e stato, dipendenti con dati anagrafici completi, ordini associati a clienti e dipendenti, e articoli acquistati. Grazie ai trigger, vengono generati automaticamente i movimenti dei punti fedeltà. **Esempio di inserimento:**

```sql
-- Inserimento di un prodotto di tipo frutta
INSERT INTO prodotto (nome, descrizione, prezzo, luogoprovenienza
   , dataraccolta, categoria, scorta)
VALUES ('Mela', 'Mela Golden', 1.20, 'Trentino', '2025-07-10', '
   FRUTTA', 50);

-- Inserimento di un dipendente
INSERT INTO dipendente (nome, cognome, codicefiscale, indirizzo,
   telefono, email)
VALUES ('Giulia', 'Bianchi', 'BNCGLI90A41H501U', 'Via Milano 5',
   '3339876543', 'giulia.bianchi@email.it');

-- Inserimento di un cliente
INSERT INTO cliente (nome, cognome, codicefiscale, indirizzo,
   telefono, email)
VALUES ('Mario', 'Rossi', 'RSSMRA80A01H501U', 'Via Roma 1', '
   3331234567', 'mario.rossi@email.it');

-- Associazione tessera al cliente
INSERT INTO tessera (codcliente, numeropunti, dataemissione,
   datascadenza, stato)
VALUES (1, 10, '2025-07-01', '2027-07-01', 'ATTIVA');

-- Inserimento di un ordine associato a cliente e dipendente
INSERT INTO ordine (prezzototale, dataacquisto, codcliente,
   coddipendente)
VALUES (0, '2025-07-29', 1, 1);

-- Inserimento di articoli nell'ordine (acquisto di 3 mele)
INSERT INTO articoliordine (codordine, codprodotto, prezzo,
   numeroarticoli)
VALUES (1, 1, 1.20, 3);
```

Questo esempio mostra un ciclo completo: creazione di prodotto, dipendente, cliente, tessera, ordine e articoli acquistati. I trigger e le funzioni implementate aggiorneranno automaticamente la scorta del prodotto, i punti della tessera e il totale dell'ordine. **Come**

utilizzare lo script:
Eseguire `Popolazione.sql` dopo la creazione delle tabelle per ottenere un database pronto all'uso, completo di dati di esempio per testare tutte le funzionalità implementate. I dati inseriti permettono di verificare le query di ricerca, le statistiche e la gestione dei punti fedeltà, simulando scenari reali di vendita e gestione clienti.